Transactions

A transaction is a set of SQL statements that execute as a unit and can be canceled if necessary. Either all the statements execute successfully, or none of them has any effect. This is achieved through the use of commit and rollback capabilities. If all of the statements in the transaction succeed, you commit it to record its effects permanently in the database. If an error occurs during the transaction, you roll it back to cancel it. Any statements executed up to that point within the transaction are undone, leaving the database in the state it was in prior to the point at which the transaction began. Commit and rollback provide the means to ensure that halfway-done operations don't make their way into your database and leave it in a partially updated (inconsistent) state.

Another use for transactions is to make sure that the rows involved in an operation are not modified by other clients while you are working with them. MySQL automatically performs locking for single SQL statements to keep clients from interfering with each other, but this is not always sufficient to guarantee that a database operation achieves its intended result, because some operations are performed over the course of several statements. In this case, different clients might interfere with each other. A transaction groups statements into a single execution unit to prevent concurrency problems that could otherwise occur in a multiple-client environment.

Using Transactions to Ensure Safe Statement Execution

Use of transactions requires a transactional storage engine such as InnoDB. Engine such as MyISAM and MEMORY will not work. By default, MySQL runs in autocommit mode, which means that changes made by individual statements are committed to the database immediately to make them permanent. In effect, each statement is its own transaction implicitly.

To perform transactions explicitly, disable autocommit mode and then tell MySQL when to commit or rollback changes. One way to perform a transaction is to issue a `START TRANSACTION` (or `BEGIN`) statement to suspend autocommit mode, execute the statements that make up the transaction, and end the transaction with a `COMMIT` statement to make the changes permanent. If an error occurs during the transaction, cancel it by issuing a `ROLLBACK` statement instead to undo the changes.

The following example illustrates this approach:

```
CREATE DATABASE trans;

use trans;
```

First, create a table to use:

```
CREATE TABLE t (name CHAR(20), UNIQUE (name)) ENGINE=InnoDB;
```

Next, initiate a transaction with START TRANSACTION, add a couple of rows to the table, commit the transaction, and then see what the table look likes:

```
START TRANSACTION;

INSERT INTO t(name) VALUES ('William');

INSERT INTO t(name) VALUES ('Wallace');
```

You can see that the rows have been recorded in the table.

```
SELECT * FROM t;
```

> Start a new instance of MySQL Command line and execute the following statement:
>
> `SELECT * FROM t;`
>
> The output is an empty set

The rows would not become visible to the second MySQL instance until the `COMMIT` statement had been issued by the first one.

In case any error occurs:

If an error occurs during a transaction, you can cancel it with ROOLBACK.

Using the table t again, you can see this by issuing the following statements:

```
START TRANSACTION;

INSERT INTO t(name) VALUES ('Gormit');

INSERT INTO t(name) VALUES ('Wallace');
```

The second `INSERT` attempts to place a row into the table that duplicates an existing name value, but fails because `name` has a `UNIQUE` index.

Execute the following statement:

```
ROLLBACK;

SELECT * FROM t;
```

After issuing the ROLLBACK, the table has only the two rows that it contained prior to the failed transaction.

In particular, the successful INSERT that was performed before the failed one has been undone, and its effect is not recorded in the table.

Issuing a START TRANSACTION statement while a transaction is in process commits the current transaction implicitly before beginning a new one.

Another way to perform transactions is to manipulate the autocommit mode directly using SET statements.

SET autocommit = 0; -- off the auto commit mode

SET autocommit = 1; -- on the auto commit mode

```
DROP TABLE t;

CREATE TABLE t (name CHAR(20), UNIQUE (name)) ENGINE=InnoDB;

SET autocommit = 0;
INSERT INTO t(name) VALUES ('William');
INSERT INTO t(name) VALUES ('Wallace');
COMMIT;
```

```
SELECT * FROM t;
```

Using Transaction Savepoints

MySQL enables you to perform a partial rollback of a transaction. To do this, issue a `SAVEPOINT` statement within the transaction to set a named marker. To roll back to just that point in the transaction later, use a `ROLLBACK` statement that names the savepoint.

The following statements illustrate how this works:

```
CREATE TABLE t2 (i INT) ENGINE=InnoDB;
START TRANSACTION;
INSERT INTO t2 VALUES (1);
SAVEPOINT my_savepoint;
INSERT INTO t2 VALUES (2);
ROLLBACK TO SAVEPOINT my_savepoint;
INSERT INTO t2 VALUES (3);
COMMIT;
SELECT * FROM t2;
```

After executing these statements, the first and third rows have been inserted, but the second one has been canceled by the partial rollback to the `my_savepoint` savepoint.

Exercise

Consider the following database for a bank that maintains Customers, Accounts, and Branches.

Customers: CustomerID, Name, City

Branches: BranchID, BranchName, Location

Accounts: AccountID, CustomerID, BranchID, Balance

Customers		
CustomerID	**Name**	**City**
1	Alice	Colombo
2	Bob	Kandy
3	Charlie	Galle
4	David	Jaffna
5	Eva	Negombo
6	Frank	Kurunegala
7	Grace	Matara
8	Hannah	Anuradhapura
9	Ishan	Batticaloa
10	Julia	Trincomalee
11	Kevin	Ratnapura
12	Lara	Badulla

Branches		
BranchID	**BranchName**	**Location**
1	Main Branch	Colombo
2	City Branch	Kandy
3	Coastal Branch	Galle
4	North Branch	Jaffna
5	Airport Branch	Negombo
6	Central Branch	Kurunegala
7	South Branch	Matara
8	Ancient Branch	Anuradhapura
9	East Branch	Batticaloa
10	Harbor Branch	Trincomalee

11	Gem Branch	Ratnapura
12	Hill Branch	Badulla

Accounts			
AccountID	**CustomerID**	**BranchID**	**Balance**
101	1	1	50000
102	2	2	30000
103	3	3	45000
104	4	4	20000
105	5	5	35000
106	6	6	60000
107	7	7	15000
108	8	8	25000
109	9	9	40000
110	10	10	55000
111	11	11	70000
112	12	12	80000
113	1	2	22000
114	3	1	33000
115	5	3	27000

a) Create the database by including all the described relations and relationships with appropriate primary and foreign key constraints.

b) Write a separate transaction for each of the following questions:

1. Write a transaction to add 5000 to AccountID = 101 and commit the transaction.
2. Write a transaction to deduct 10000 from AccountID = 102 and rollback the transaction instead of committing.
3. Write a transaction to transfer 7000 from AccountID = 103 to AccountID = 104 and commit only if both operations succeed.
4. Write a transaction to deduct 5000 from AccountID = 105, create SAVEPOINT sp1, deduct 3000 from AccountID = 106, Rollback to sp1, and commit.
5. Write a transaction to add 2000 to AccountID = 107, SAVEPOINT a, add 3000 to AccountID = 108, SAVEPOINT b, add 4000 to AccountID = 109, Rollback to SAVEPOINT a, and Commit.

6. Write a transaction to deduct 2000 from AccountID = 110, SAVEPOINT before_second, Deduct 3000 from the same account, Rollback to before_second, and Commit.
7. Write a transaction to deduct 10000 from AccountID = 111. If balance becomes negative, then rollback; else commit.
8. Write a transaction to transfer 15000 from AccountID = 112 to AccountID = 101, commit only if the sender has a sufficient balance.
9. Write a transaction to Deposit 1000 into 3 accounts (IDs: 102, 103, 104), use SAVEPOINT before each update, and Rollback only the last deposit.
10. Write a transaction to deduct 5000 from AccountID = 105, deduct 5000 from AccountID = 106, add 10000 to AccountID = 107, and commit only if all succeed.
11. Write a transaction to add 3000 to AccountID = 108, introduce an error (e.g., invalid AccountID update), and rollback the entire transaction.
12. Write a transaction to update AccountID = 109 balance, SAVEPOINT safe_point, perform another update, rollback to safe_point if error occurs, and commit.
13. Write a transaction to deduct 8000 from AccountID = 110, record payment in another table (assume exists), and commit if both succeed.
14. Write a transaction to reverse a transfer (add to sender, deduct from receiver), use SAVEPOINT before reversal, and rollback if any issue.
15. Write a transaction to deduct 10000 from AccountID = 111, SAVEPOINT step1, add 6000 to AccountID = 112, SAVEPOINT step2, add 4000 to AccountID = 101, rollback to step1, and commit remaining operations.